Questions and Comprehensive Answers

TECHNICAL PROBLEM SETS & ANALYTICAL SOLUTIONS

1 HTTP Response Analysis

```
HTTP/1.1 200 OK Date: Mon, 01 Apr 2026 10:00:00 GMT Content-Type: text/html Content-Leng
th: -150 Connection: keep-alive
```

1. Identify any incorrect or invalid fields in the header:

The primary invalid field is `Content-Length: -150`. Under RFC 9110 (HTTP Semantics), the `Content-Length` header field must be a non-negative decimal integer representing the octet length of the payload body. A negative value is strictly illegal. Additionally, in the raw snippet provided, missing standard CRLF line breaks between header lines would cause parsing failure if sent without delimiter separation.

2. Analyze the issue with Content-Length and explain its impact:

Because `Content-Length` controls how many bytes the client or downstream proxy reads from the TCP socket for a persistent connection (`Connection: keep-alive`), a negative integer corrupts stream framing. The browser parser cannot determine payload boundaries, triggering request/response desynchronization, protocol parsing errors (e.g., `ERR_INVALID_HTTP_RESPONSE`), premature connection closure, or potential HTTP request smuggling vulnerabilities on intermediate proxies.

3. Determine whether the status code matches the response correctness:

No. Status code `200 OK` indicates successful processing and valid representation delivery. Transmitting an unparseable or contradictory header alongside a success code creates an invalid protocol state. If the server encountered an internal formatting error, it should transmit `500 Internal Server Error`, or if rejecting invalid inbound data, a `4xx` status.

4. Suggest corrections for the identified errors:

Replace the negative value with the actual byte size of the HTML payload (e.g., `Content-Length: 150`), or omit `Content-Length` entirely and use chunked transfer encoding (`Transfer-Encoding: chunked`) for dynamic payloads.

2 DOM Structure Analysis

```
<html> <head> <title>TestPage</title> </head> <body> <div> <p>HelloWorld </div> </body>
</html>
```

1. Identify missing or improperly closed elements:

The paragraph element `<p>` is opened but never explicitly closed with a corresponding `</p>` tag before the parent `<div>` is closed.

2. Analyze how the browser will interpret this structure:

Modern HTML5 tree construction algorithms specify automatic end-tag insertion rules for optional closing tags. When the parser encounters the closing tag `</div>` while inside an open `<p>` formatting context, it executes the "generate implied end tags" rule. It automatically closes the `<p>` element immediately before closing the `<div>`.

3. Determine the impact on rendering and layout:

In standard visual layout, the text "HelloWorld" will render correctly within paragraph boundaries. However, unclosed elements introduce ambiguity into CSS cascading rules, JavaScript DOM traversing scripts (e.g., unexpected sibling or child query results), and layout calculation overhead during document tree synthesis.

4. Suggest corrections to make the DOM valid:

Add the missing closing `</p>` tag explicitly: `<div><p>HelloWorld</p></div>`.

3 GET vs POST in Login System

Method	URL Example	Data Visibility	Security Level
GET	`/login?user=admin&pass=1234`	Visible in URL query string	Low
POST	`/login`	Hidden in HTTP message body	Moderate

1. Interpret the differences in data transmission between GET and POST:

GET encodes payload data directly into the Uniform Resource Identifier (URI) query string, making it an idempotent and safe method designed purely for resource retrieval. **POST** transmits data within the HTTP request entity body, designed for non-idempotent state changes, submissions, and mutations without polluting the URI path.

2. Analyze which method is more suitable for login and why:

POST is the required method for authentication. Authentication initiates session creation or credential verification (a stateful operation). POST conceals sensitive credentials from address bars, bookmarks, web caches, and link-sharing workflows.

3. Identify security risks in using GET for authentication:

Using GET exposes plaintext credentials across multiple vectors: browser history, forward/backward cache, proxy and web server access logs (e.g., Nginx access logs storing full URIs), shoulder surfing, and the HTTP `Referer` header leaked to external resources embedded on subsequent pages.

4. Suggest best practices for secure login implementation:

Enforce HTTPS (TLS) to encrypt payload in transit; submit credentials via POST; protect against CSRF using anti-CSRF tokens or SameSite cookies; hash passwords on the backend using Argon2 or bcrypt; implement rate limiting; and store session references in secure, `HttpOnly`, `SameSite=Strict` cookies.

4 HTML Code Inspection

```
<!DOCTYPE html><html><head><title>Sample</title></head><body><h1>Welcome<p>This is a par
agraph<img src="image.jpg"></body></html>
```

1. Identify missing closing tags:

Missing tags include: the closing heading tag `</h1>` and the closing paragraph tag `</p>`.

2. Analyze issues with the <img> tag:

The `<img>` element lacks an `alt` attribute (mandatory for WCAG accessibility and fallback screen reader consumption), lacks explicit `width` and `height` dimensions (causing Cumulative Layout Shift / CLS during asset loading), and lacks a lazy loading directive.

3. Determine how browsers will handle these errors:

Under HTML5 error recovery, the opening <p> tag terminates the preceding <h1> automatically because headings cannot contain block elements. Similarly, encountering `</body>` closes the unterminated <p> tag. The `<img>` is treated as a void element and requires no end tag in HTML5. While browsers patch these mistakes, relying on parser heuristics slows down DOM tree construction.

4. Suggest a corrected version of the code:

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <title>Sample</title> </h
ead> <body> <h1>Welcome</h1> <p>This is a paragraph</p> <img src="image.jpg" alt="Descriptive s
ample image" width="600" height="400"> </body> </html>
```

5 Browser-Server Communication Flow

Given Communication Sequence:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

1. Interpret each step in the communication flow:

Step 1 (URL Entry): The user inputs an address (e.g., `https://example.com`); the browser parses the protocol, host, and port.
Step 2 (DNS Lookup): The client queries hierarchical domain name servers to resolve the human-readable domain name into a routable IP address.
Step 3 (HTTP Request Sent): After establishing a TCP connection (and completing a TLS handshake if HTTPS), the browser dispatches an HTTP request message detailing the method, headers, and body.
Step 4 (Server Processing): The web/application server authenticates, executes business logic, queries database storage, and generates a formatted payload.
Step 5 (Response Returned): The server responds with status codes, response headers, and the resource payload (HTML, JSON, asset streams).
Step 6 (Browser Rendering): The browser engine tokenizes HTML to construct the DOM, processes CSS for the CSSOM, builds the Render Tree, performs layout/reflow calculations, and paints pixels to screen.

2. Identify where delays can occur:

Delays emerge across four major bottlenecks: (a) *Network Latency:* High DNS lookup latency, multi-round-trip TCP/TLS handshakes, and geographic distance; (b) *Server Processing:* Unoptimized database queries, unindexed searches, synchronous third-party API blocking, and inadequate compute resources; (c) *Bandwidth & Transfer:* Uncompressed payloads causing high Time to First Byte (TTFB) or slow download speeds; (d) *Client-side Rendering:* Parser-blocking external CSS/JS scripts, large unoptimized DOM trees, and unoptimized high-resolution images.

3. Analyze the role of DNS in the process:

DNS serves as the foundational address resolution layer of the internet. It maps human-friendly hostnames to machine-actionable IPv4/IPv6 addresses. Without rapid and accurate DNS resolution, all downstream TCP connection handshakes and HTTP request transactions are completely blocked.

4. Suggest optimizations to improve performance:

• **DNS & Connection:** Implement DNS prefetching (`<link rel="dns-prefetch">`), utilize Anycast global DNS providers, enable HTTP/3 (QUIC) for 0-RTT connection resumption.
• **Edge & Server Caching:** Deploy Content Delivery Networks (CDNs) to serve cached static resources close to edge users; optimize database queries and implement server caching (Redis/Memcached).
• **Compression & Transport:** Compress assets with Brotli or Gzip, utilize HTTP/2 or HTTP/3 multiplexing to eliminate head-of-line blocking.
• **Render Tree Optimization:** Defer non-critical JavaScript (`defer/async`), inline critical CSS, specify image dimensions, and use modern responsive image formats (WebP/AVIF).